BEDHOPPER

Full Development Plan Using Antigravity

Detailed step-by-step blueprint for building the BedHopper ultra-low-cost accommodation platform

BUILD THE LOWEST-COST SAFE BED MARKETPLACE

PWA + Authentication + PostgreSQL + Payments + Maps + Trust

Platform:	Antigravity / AI visual development environment
Product:	BedHopper
Target:	Budget travelers, hosts, hostels and institutions
MVP Target:	Approximately 40 development days
Document Date:	August 2026

Implementation blueprint. Validate Antigravity's actual capabilities before relying on any platform-specific feature.

Contents

1 Project Overview

BedHopper is an open-source, ultra-low-cost accommodation marketplace designed to help travelers find the cheapest legitimate place to sleep.

The platform combines:

- Peer-to-peer low-cost sleeping spaces.
- Budget hostels and guesthouses.
- Optional Service-Share accommodation.
- Trust and identity verification.
- Search, booking, payments and messaging.
- Open-source self-hosting.

The first version should focus on a narrow, reliable MVP rather than attempting to implement every global marketplace feature at launch.

Primary MVP goal: prove that verified low-cost supply can be discovered, booked and supported safely.

2 Antigravity Assumptions

This development plan assumes Antigravity provides some or all of the following capabilities:

- Visual UI builder.
- Database or collection management.
- Authentication.
- Workflow and business logic.
- API integrations.
- File uploads.
- Responsive web application support.
- PWA or service-worker support.
- Deployment or code export.

If a specific capability is unavailable, replace it with a conventional backend service or generated application code.

Important:

Antigravity's exact capabilities should be verified before implementation. Do not design critical architecture around a feature that the platform does not actually support.

3 Architecture Overview

The proposed high-level architecture is:

4 Phase 0 – Preparation

Days 1–2
Define the data model, project structure, authentication strategy, payment architecture and environment configuration before building the user interface.

4.1 0.1 Users Collection

Create a `Users` collection/table.

Field	Description
id	Unique user identifier
name	Display name
email	Login email
password_hash	Managed by authentication system
profile_photo	Profile image URL
bio	Short profile description
phone	Optional verified phone number
id_verified	Verification status
role	traveler / host / both / admin
created_at	Account creation timestamp

4.2 0.2 Listings Collection

Fields:

- `host_id`
- `title`
- `description`
- `type`
- `max_guests`
- `price_per_night`
- `currency`
- `city`
- `lat`
- `lng`

- address
- photos
- amenities
- house_rules
- check_in_time
- check_out_time
- is_service_share
- service_description
- service_hours_per_day
- available_dates
- active
- created_at

4.3 0.3 Bookings Collection

- listing_id
- guest_id
- check_in_date
- check_out_date
- total_price
- platform_fee
- status
- is_service_share
- service_agreed
- created_at

4.4 0.4 Reviews Collection

- booking_id
- reviewer_id
- reviewee_id
- rating
- comment
- created_at

4.5 0.5 Service-Share Agreements

Where required:

- `booking_id`
- `tasks_agreed`
- `hours_agreed`
- `deposit_held`
- `status`

4.6 0.6 Project Setup

1. Create the Antigravity project.
2. Select responsive web application / PWA architecture.
3. Configure authentication.
4. Configure development, staging and production environments.
5. Configure PostgreSQL or the platform's database.
6. Configure environment secrets.
7. Configure payment integration.

5 Phase 1 – User System and Profiles

Days 3–5
Build authentication, onboarding, identity status and the initial traveler/host dashboard.

5.1 1. Sign-up and Login

Create:

- Sign-up screen.
- Login screen.
- Forgot-password flow.
- Email verification.
- Optional social login.

After authentication, redirect the user to onboarding.

5.2 2. Onboarding

Three initial steps:

1. Profile photo and biography.
2. Select role: traveler, host or both.

3. Identity verification status.

Do not store sensitive identity documents unnecessarily.

If identity verification is required, prefer a specialized verification provider and retain only the minimum verification result needed.

5.3 3. User Dashboard

Show:

- Upcoming bookings.
- Pending booking requests.
- Create Listing.
- Search Beds.
- Profile verification status.
- Messages.

6 Phase 2 – Host Listing System

Days 6–10
Build a guided listing wizard and host management dashboard.

6.1 4. Become a Host Wizard

Page 1 – Listing Type

- Private room.
- Shared room.
- Couch / shared space.
- Service-Share bed.

Page 2 – Photos

- Drag-and-drop upload.
- Image compression.
- Maximum file-size validation.
- Primary image selection.

Page 3 – Details

- Title.
- Description.
- Maximum guests.
- Price per night.

- Currency.
- City.
- Address.
- Map location.

Page 4 – Amenities and Rules

- Wi-Fi.
- Kitchen.
- Bathroom.
- Air conditioning.
- Laundry.
- House rules.

Page 5 – Availability

Create a calendar allowing hosts to:

- Open dates.
- Block dates.
- View bookings.

Page 6 – Service-Share

If enabled:

- Service description.
- Expected hours.
- Task category.
- Restrictions.

Allow:

- Save Draft.
- Preview.
- Publish.

6.2 5. Host Dashboard

Include:

- Active listings.
- Draft listings.
- Booking requests.
- Calendar.

- Earnings.
- Messages.
- Verification status.

7 Phase 3 – Traveler Search and Booking

Days 11–16
Build the core marketplace experience: search, filtering, listing details, booking and messaging.

7.1 6. Homepage Search

Primary search component:

Where are you going?

Inputs:

- Destination.
- Check-in.
- Check-out.
- Guests.

CTA:

Search Beds

7.2 7. Search Results

Provide:

- Map view.
- List view.
- Price filter.
- Property-type filter.
- Amenities filter.
- Verified-host filter.

Default sorting:

$$LowestTotalPriceFirst$$

Each result card should show:

- Thumbnail.
- Title.
- Price per night.

- Estimated total price.
- Rating.
- Host verification.
- Property type.

7.3 8. Listing Details

Include:

- Photo gallery.
- Description.
- Amenities.
- House rules.
- Host profile.
- Verification badge.
- Response rate.
- Availability calendar.
- Booking CTA.

Service-Share listings should display task requirements clearly before the traveler submits an application.

7.4 9. Booking Flow

1. Select dates.
2. Confirm guests.
3. Calculate price.
4. Show platform fee.
5. Confirm policies.
6. Process payment.
7. Create booking.
8. Notify host.
9. Display confirmation.

For paid bookings:

$$Total = (NightlyPrice \times Nights) + PlatformFee + ApplicableTaxes$$

Taxes and fees should be calculated according to the applicable jurisdiction rather than hard-coded globally.

7.5 10. Messaging

Create a `Messages` collection.

Basic functionality:

- Guest-to-host messages.
- Booking-linked conversations.
- Message timestamps.
- Report/block controls.
- Moderation hooks.

8 Phase 4 – Hostel / Hotel Integration

Days 17–20
Add commercial accommodation inventory and subscription monetization.

8.1 11. Commercial Host Onboarding

Allow:

I represent a hostel / hotel / guesthouse

Additional fields:

- Business name.
- Registration information.
- Property address.
- Property photos.
- Business contact.

Require manual approval initially.

8.2 12. Property and Bed Inventory

Introduce a `Properties` entity:

$$Property \rightarrow MultipleListings$$

Example:

Hostel → 6-bed dorm → 8-bed dorm → private room

8.3 13. Budget Price Rules

Create a configuration table for budget thresholds.

The system can display:

- Budget badge.

- Price comparison.
- Warning when price exceeds target range.

Do not falsely claim that a price is globally “cheap” without considering city, dates, taxes and market conditions.

8.4 14. Hostel Subscription

Initial hypothesis:

$$\$20/month$$

Potential benefits:

- Unlimited listings.
- Reduced/no per-booking platform fee.
- Analytics.
- Promotional tools.

9 Phase 5 – Service-Share

Days 21–24
Introduce Service-Share only as a controlled feature with explicit safety, legal and operational safeguards.

9.1 15. Service-Share Listing

Host enters:

- Task description.
- Expected hours.
- Accommodation details.
- Eligibility requirements.
- Prohibited tasks.

9.2 16. Application Flow

1. Traveler opens listing.
2. Traveler reviews tasks.
3. Traveler submits application.
4. Host reviews application.
5. Host approves/rejects.
6. Agreement is created.
7. Deposit, if legally and operationally appropriate, is handled.

9.3 17. Service-Share Pass

Potential pricing:

$$\$3/month$$

Potential benefits:

- Verified Service Guest badge.
- Priority support.
- Additional trust features.

A paid pass should never imply that the user is exempt from legal or safety requirements.

10 Phase 6 – Trust, Safety and Reviews

Days 25–28
Build verification, reviews, reporting and administrative safety controls.

10.1 18. Identity Verification

Possible approaches:

- Manual admin verification.
- Specialized identity-verification provider.
- Platform-managed verification status.

Store the minimum necessary information.

10.2 19. Review System

After a completed booking:

$$\text{Guest Review} \leftrightarrow \text{Host Review}$$

Rules:

- One review per party per booking.
- Reviews linked to completed bookings.
- Abuse reporting.
- Moderation workflow.

10.3 20. Safety Features

Every listing and profile should provide:

- Report button.
- Block functionality.
- Safety policy.
- Admin escalation.

11 Phase 7 – Payments and Monetization

Days 29–33
Implement payments, host payouts, platform fees and subscriptions.

11.1 21. Stripe Connect

Conceptual flow:

$$TravelerPayment \rightarrow Platform \rightarrow PlatformFee + HostPayout$$

Exact fund flows, taxes, refunds and payout timing must follow the payment provider's supported marketplace architecture.

11.2 22. Booking Fee

Initial hypothesis:

$$8\%$$

Display the fee transparently before payment.

11.3 23. Subscriptions

Potential plans:

- Hostel subscription.
- Service-Share Pass.
- Optional future verification plans.

Benefits should automatically activate only after successful subscription payment.

12 Phase 8 – Admin Dashboard

Days 34–36
Build the control center for users, listings, bookings, reports and revenue.

12.1 24. Admin Panel

The administrator should be able to:

- View users.
- Suspend users.
- Review verification requests.
- Approve commercial hosts.
- View listings.

- Remove unsafe listings.
- View bookings.
- Handle disputes.
- Review reports.
- View revenue metrics.
- View system audit logs.

12.2 Admin Roles

At minimum:

- Super Admin.
- Trust and Safety Admin.
- Operations Admin.
- Support Admin.

Apply role-based access control so support users cannot perform financial or destructive administrative operations without authorization.

13 Phase 9 – Open Source and Deployment

Days 37–40
Prepare the repository, documentation, production deployment and PWA.

13.1 25. Open-source Repository

If Antigravity supports code export:

1. Export source code.
2. Create GitHub repository.
3. Add MIT license.
4. Add README.
5. Add environment-variable documentation.
6. Add installation instructions.
7. Add database setup instructions.
8. Add contribution guidelines.

If Antigravity does not support code export, publish:

- Architecture documentation.
- Data model.
- UI specifications.

- API documentation.
- Deployment instructions.
- “Build BedHopper on Antigravity” guide.

13.2 26. Production Deployment

Production checklist:

- Custom domain.
- HTTPS / SSL.
- Production database.
- Secret management.
- Error monitoring.
- Database backups.
- Logging.
- Rate limiting.
- Security headers.
- Payment webhooks.

13.3 27. PWA

Configure:

- Web app manifest.
- Service worker.
- Add-to-home-screen support.
- Responsive layouts.
- Offline caching for safe non-sensitive content.

Do not cache sensitive booking, payment or identity information in insecure client storage.

14 Phase 10 – Launch and Marketing

14.1 28. Pilot Cities

Start with two cities.

Example candidates:

- Bangkok.
- Mexico City.

The final cities should be selected using:

- Accommodation affordability.

- Tourist volume.
- Hostel density.
- Regulatory feasibility.
- Payment availability.
- Local operating capability.

Manually onboard an initial supply of:

$$10 - -20 hosts/properties per pilot city$$

before spending heavily on user acquisition.

14.2 29. Content Marketing

Create:

- "Cheapest beds in Bangkok"
- "Cheapest beds in Mexico City"
- Budget travel guides.
- Hostel comparison pages.
- Safety guides.

14.3 30. Community Building

Potential communities include:

- Budget travel communities.
- Backpacking communities.
- Digital nomad communities.
- Open-source developers.
- University groups.

Example launch message:

Show HN: BedHopper – Open-source ultra-low-cost accommodation marketplace

15 Core User Flows

15.1 Traveler Flow

$$Landing Page \rightarrow Search \rightarrow Filters \rightarrow Listing \rightarrow Booking \rightarrow Payment \rightarrow Stay \rightarrow Review$$

15.2 Host Flow

$$Sign Up \rightarrow Verification \rightarrow Create Listing \rightarrow Publish \rightarrow Booking \rightarrow Stay \rightarrow Payout \rightarrow Review$$

15.3 Service-Share Flow

$$Listing \rightarrow Application \rightarrow HostApproval \rightarrow Agreement \rightarrow Stay \rightarrow Completion \rightarrow Review$$

16 MVP Security Requirements

Security should be included from the beginning.

- Never store raw passwords.
- Use secure authentication.
- Protect API endpoints.
- Validate all user input.
- Apply authorization server-side.
- Use role-based permissions.
- Protect payment webhooks.
- Verify webhook signatures.
- Rate-limit authentication and messaging endpoints.
- Keep secrets outside source code.
- Maintain audit logs for sensitive admin actions.
- Back up the production database.
- Minimize storage of identity documents.
- Encrypt sensitive data where appropriate.
- Implement account deletion and data-retention policies.

17 Database Relationships

The core relationships are:

$$User \rightarrow Listings$$

$$Listing \rightarrow Bookings$$

$$Booking \rightarrow Reviews$$

$$Booking \rightarrow Messages$$

$$Booking \rightarrow Service - ShareAgreement$$

For commercial inventory:

$$Business \rightarrow Property \rightarrow Listings$$

This structure allows BedHopper to grow from individual hosts to professional accommodation operators.

18 40-Day MVP Timeline

Phase	Days	Focus
Phase 0	1–2	Preparation
Phase 1	3–5	Authentication and profiles
Phase 2	6–10	Listings and host dashboard
Phase 3	11–16	Search, booking and messaging
Phase 4	17–20	Hostel integration
Phase 5	21–24	Service-Share
Phase 6	25–28	Trust, safety and reviews
Phase 7	29–33	Payments and monetization
Phase 8	34–36	Admin dashboard
Phase 9	37–40	Deployment and open-source release

40 days is an aggressive MVP target. Safety, payments, identity verification and marketplace operations should take priority over feature count.

19 Definition of Done

The MVP should not be considered complete merely because every screen exists.

The MVP is ready for pilot launch when:

1. A new traveler can create an account.
2. A host can create and publish a listing.
3. A traveler can search and filter listings.
4. Availability is correctly enforced.
5. A traveler can complete a booking.
6. Payment and webhook handling work reliably.
7. The host receives the booking.
8. Messaging works.
9. Reviews can be submitted after completion.
10. Users can report unsafe behavior.
11. Admins can suspend accounts and listings.
12. Database backups are working.
13. Production secrets are protected.

14. Basic monitoring and logging are active.
15. The platform has clear Terms, Privacy and Safety policies.

20 Post-MVP Roadmap

After the first pilot:

- Improve search ranking.
- Add multilingual support.
- Add additional payment methods.
- Add stronger fraud detection.
- Add host analytics.
- Add traveler loyalty and referrals.
- Add verified property checks.
- Add white-label deployments.
- Expand city by city.
- Explore federation between self-hosted instances.

21 Conclusion

BedHopper should be built as a focused marketplace rather than as a collection of disconnected features.

The correct sequence is:

$$Supply \rightarrow Trust \rightarrow Search \rightarrow Booking \rightarrow Payments \rightarrow Retention \rightarrow Scale$$

The first milestone is not:

Build every feature.

The first milestone is:

Can a real traveler safely find and book a genuinely low-cost bed?

If the answer is yes, BedHopper can progressively add:

- Hostel subscriptions.
- Service-Share.
- Verification products.
- White-label deployments.
- International expansion.
- Federated open-source infrastructure.

BEDHOPPER — FULL DEVELOPMENT PLAN

Antigravity Implementation Blueprint

August 2026